

OAK-D 엣지 AI 실험 기록

Step 01 · 공식 예제 실행 · 2026-09-16 · OAK-D PoE · depthai 3.10.0

1. 엣지에 모델 올리기

원리

호스트는 정의와 결과 수신만 한다. 추론은 카메라 안의 칩에서 돈다.

호스트	파이프라인 정의 → 모델과 함께 장치로 업로드
장치	카메라 → 리사이즈 → 신경망(NN) 추론 → 디코딩 · NMS
호스트	검출 박스 수신 → 화면 표시

방법

```
pip install depthai opencv-python
```

```
import depthai as dai, cv2

with dai.Pipeline() as pipeline:
    cam = pipeline.create(dai.node.Camera).build()
    nn = pipeline.create(dai.node.DetectionNetwork).build(
        cam, dai.NNModelDescription("yolov6-nano"))

    qRgb = nn.passthrough.createOutputQueue()    # 영상
    qDet = nn.out.createOutputQueue()            # 검출 결과

    pipeline.start()

    while pipeline.isRunning():
        frame = qRgb.get().getCvFrame()
        h, w = frame.shape[:2]
        for d in qDet.get().detections:
            cv2.rectangle(frame, (int(d.xmin*w), int(d.ymin*h)),
                           (int(d.xmax*w), int(d.ymax*h)), (255,0,0), 2)
```

```
cv2.imshow("rgb", frame)
if cv2.waitKey(1) == ord("q"):
    break
```

코드	하는 일
NNModelDescription	모델 지정. 자동 다운로드 후 <code>~/.cache/depthai/models/</code> 에 캐시
createOutputQueue	장치 → 호스트 통로. 연 것만 전송된다
pipeline.start()	업로드 + 장치 부팅. 몇 초 걸린다
d.xmin ~ d.ymax	박스 좌표. 0~1 비율 이라 프레임 크기를 곱해야 한다
nn.getClasses()	클래스 이름 목록

2. 입력 → 결과

신경망 단독 처리 용량

입력				출력	
모델	해상도	픽셀 수	디코딩	FPS	픽셀 처리량
YOLOv6n 예제	512×288	147,456	칩 위	38.30	5.65 Mpx/s
YOLOv6n	512×288	147,456	호스트	40.21	5.93 Mpx/s
YOLOv6n 해상도 비교	512×384	196,608	칩 위	22.56	4.44 Mpx/s
YOLO26n	512×288	147,456	호스트	18.00	2.65 Mpx/s
YOLOv10n	512×288	147,456	호스트	16.92	2.49 Mpx/s

카메라 → 신경망 실제 경로

입력				출력		
모델	해상도	디코딩	카메라 FPS	FPS	지연	용량 여유
YOLOv6n 예제	512×288	칩 위	30	30.04	미측정	+28%

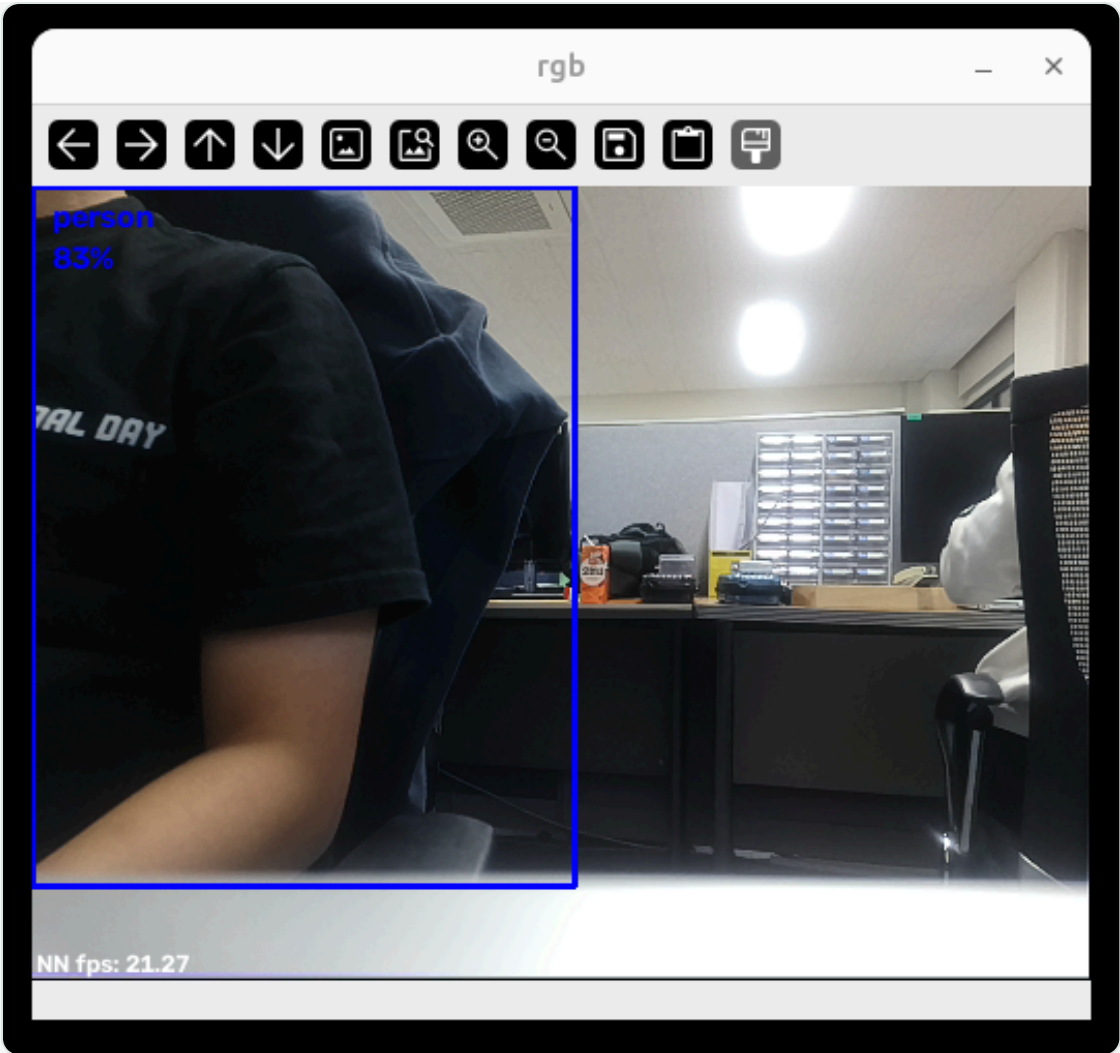
입력				출력		
모델	해상도	디코딩	카메라 FPS	FPS	지연	용량 여유
YOLOv6n	512×288	칩 위	30	30.01	51.00 ms	+28%
YOLOv6n	512×288	호스트	30	30.01	49.04 ms	+34%
YOLO26n	512×288	호스트	30	17.08	464.80 ms	-40%
YOLOv10n	512×288	호스트	30	16.14	492.29 ms	-44%
YOLOv10n	512×288	호스트	15	15.00	83.11 ms	+13%

용량 여유가 음수면 카메라가 모델보다 빨라 프레임이 쌓이고 지연이 급증한다. 같은 YOLOv10n을 30fps로 밀면 492ms, 15fps로 낮추면 83ms다.

지연은 장치 안에서 켜 값(촬영 → 신경망 출력)이다. 호스트 전송·디코딩 시간은 포함되지 않아, 칩 위(51.00ms)가 호스트(49.04ms)보다 디코딩·NMS만큼 길게 나온다.

3. 예제 모델과 실행 결과

항목	값
모델	YOLOv6n — 객체 검출
클래스	COCO 80종 (사람, 차, 신호등 등)
입력	512×288 — 변형을 명시해 지정 (기본값은 512×384)
출력	박스 좌표 + 클래스 + 확신도
디코딩	칩 위 — 중복 박스 제거(NMS)까지 장치가 처리



사람을 person 83% 로 검출. 좌하단은 예제가 계산한 FPS.

분석

측정 대상	FPS	비고
신경망 단독	38.30	추론만. 칩의 상한
예제 전체	30.04	카메라 30fps에 막힘